

Estratégias de Atualização e Prometheus

O que você aprenderá nesta página

Comparar estratégias de atualização Implantar software com lançamentos canário (*canary releases*) Usar o Prometheus para consultas personalizadas

No capítulo anterior, realizamos atualizações automatizadas com um pipeline de implantação. A atualização simplesmente funcionou, mas não fazemos ideia de como ela realmente ocorreu, a não ser pelo fato de que um *pod* foi alterado. Vamos agora analisar como podemos tornar o processo de atualização mais seguro, ajudando-nos a alcançar um número maior de "noves" (*nines*) na disponibilidade.

Existem múltiplas estratégias de atualização/implantação/lançamento. Vamos nos concentrar em duas delas:

Atualização contínua (*Rolling update*) Lançamento canário (*Canary release*)

Ambas as estratégias de atualização são projetadas para garantir que a aplicação continue funcionando durante e após uma atualização. Em vez de atualizar todos os *Pods* ao mesmo tempo, a ideia é atualizá-los um de cada vez e confirmar se a aplicação está funcionando.

Atualização contínua (*Rolling update*)

Por padrão, o Kubernetes inicia uma atualização contínua (*rolling update*) quando alteramos a imagem. Isso significa que cada *pod* é atualizado sequencialmente. A atualização contínua é um excelente padrão, pois permite que a aplicação permaneça disponível durante o processo de atualização. Se decidirmos enviar uma imagem que não funciona, a atualização será interrompida automaticamente.

Preparei uma aplicação e imagens com 5 versões.

A versão com a tag v1 sempre funciona v2 nunca funciona v3 funciona 90% das vezes v4 morrerá após 60 segundos, e v5 sempre funciona

A seguinte configuração pode ser usada para colocar a aplicação v1 em funcionamento:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: flaky-update-dep
spec:
  replicas: 4
  selector:
    matchLabels:
      app: flaky-update
  template:
    metadata:
      labels:
        app: flaky-update
    spec:
      containers:
        - name: flaky-update
          image: mluukkai/dwk-app8:v1

---

apiVersion: v1
kind: Service
metadata:
```

```

    name: flaky-update-svc
spec:
  selector:
    app: flaky-update
  ports:
    - protocol: TCP
      port: 80
      targetPort: 3541
  type: LoadBalancer

```

Vamos implantar a aplicação:

```

$ kubectl apply -f deployment.yaml
deployment.apps/flaky-update-dep created

```

```

$ kubectl get po

```

NAME	READY	STATUS	RESTARTS	AGE
flaky-update-dep-7b5fd9ffc7-27cxt	1/1	Running	0	87s
flaky-update-dep-7b5fd9ffc7-mp8vd	1/1	Running	0	88s
flaky-update-dep-7b5fd9ffc7-m4smm	1/1	Running	0	87s
flaky-update-dep-7b5fd9ffc7-nzl98	1/1	Running	0	88s

Como podemos ver, a aplicação está funcionando. Agora mude a tag para v2 e aplique.

```

$ kubectl apply -f deployment.yaml
$ kubectl get po --watch

```

NAME	READY	STATUS	RESTARTS	AGE
flaky-update-dep-84f4c94bc7-dkqcw	1/1	Running	0	17s
flaky-update-dep-84f4c94bc7-h7d2t	1/1	Running	0	9s

Você pode ver que a atualização contínua foi concluída, mas infelizmente, a aplicação não funciona mais. A aplicação está em execução, apenas existe um bug que a impede de funcionar corretamente. Como comunicamos esse mau funcionamento fora da aplicação? É aqui que entram as *ReadinessProbes*.

Com uma *ReadinessProbe*, o Kubernetes pode verificar se um *pod* está pronto para processar requisições. A aplicação possui um *endpoint* `/healthz` na porta 3541, e podemos usá-lo para testar a integridade. Ele simplesmente responderá com o código de status 500 se não estiver funcionando e 200 se estiver.

Vamos reverter a versão para a v1 também, para que possamos testar a atualização para a v2 novamente.

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: flaky-update-dep
spec:
  replicas: 4
  selector:
    matchLabels:
      app: flaky-update
  template:
    metadata:
      labels:
        app: flaky-update
    spec:
      containers:
        - name: flaky-update
          image: mluukkai/dwk-app8:v1
          readinessProbe:
            initialDelaySeconds: 10 # Atraso inicial ate que a prontidao seja testada
            periodSeconds: 5 # Com que frequencia testar
            httpGet:
              path: /healthz
              port: 3541

```

Aqui, o `initialDelaySeconds` e o `periodSeconds` significam que a sonda é enviada 10 segundos depois que o contêiner estiver ativo e a cada 5 segundos depois disso. Agora, se alterarmos a tag para v2 e aplicarmos, três dos *Pods* estarão completamente funcionais. Um dos *Pods* v1 foi removido para dar lugar aos v2, mas como eles não funcionam, nunca ficam com status *READY*, e a atualização não pode continuar.

Continuando o curso com k3d

Você pode continuar usando o Google Kubernetes Engine ou voltar a usar o k3d. Como nossas aplicações agora usam a Gateway API, você deve habilitá-la no cluster k3d.

Crie uma *ReadinessProbe* para a aplicação Ping-pong. Ela deve estar pronta quando tiver uma conexão com o banco de dados. E crie outra *ReadinessProbe* para a aplicação *Log output*. Ela deve estar pronta quando puder receber dados da aplicação Ping-pong.

Crie as sondas (*probes*) e o *endpoint* necessários para O Projeto para garantir que ele esteja funcionando e conectado a um banco de dados. Adicione um botão à sua aplicação que pode ser usado para "quebrar" a aplicação.

```
let isHealthy = true
// ...
app.get("/healthz", (req, res) => {
  if (!isHealthy) {
    return res.status(500).json({ status: "unhealthy" })
  }
  return res.status(200).json({ status: "ok" })
})
```

Lançamento canário (*Canary release*)

Com atualizações contínuas, ao incluir as Sondas (*Probes*), poderíamos criar lançamentos sem tempo de inatividade para os usuários. Às vezes isso não é o suficiente e você precisa ser capaz de fazer um *lançamento parcial* para alguns usuários e obter dados de uso e estatísticas para o próximo lançamento. *Lançamento canário* é o termo usado para descrever uma estratégia de lançamento em que introduzimos um subconjunto dos usuários a uma nova versão da aplicação. Quando a confiança no novo lançamento aumenta, o número de usuários na nova versão pode ser aumentado até que a versão antiga não seja mais usada.

Usaremos o Argo Rollouts para testar um tipo de lançamento canário:

```
$ kubectl create namespace argo-rollouts
$ kubectl apply -n argo-rollouts -f https://github.com/argoproj/argo-rollouts/releases/
latest/download/install.yaml
```

O *Rollout* substituirá nosso *deployment* criado anteriormente e nos permitirá usar um novo campo:

```
apiVersion: argoproj.io/v1alpha1
kind: Rollout
metadata:
  name: flaky-update-dep
spec:
  replicas: 4
  strategy:
    canary:
      steps:
        - setWeight: 25
        - pause:
            duration: 30s
        - setWeight: 50
        - pause:
            duration: 30s
# ... Sondas omitidas por brevidade
```

A estratégia acima moverá primeiro 25% (*setWeight*) dos *Pods* para uma nova versão, após o que esperará por 30 segundos, moverá para 50% dos *Pods* e depois aguardará por 30 segundos até que cada *Pod* seja atualizado.

Inicie agora o Prometheus com Helm e use *port-forward* para acessar o site GUI. Escreva uma consulta que mostre o número de *Pods* criados por *StatefulSets* no *namespace* *prometheus*.

Com o novo *Rollout* e *AnalysisTemplate*, podemos testar com segurança a implantação de qualquer versão. O *AnalysisTemplate* usará o Prometheus para consultar o estado do *deployment*.

```
apiVersion: argoproj.io/v1alpha1
kind: AnalysisTemplate
metadata:
  name: restart-rate
spec:
  metrics:
    - name: restart-rate
      initialDelay: 2m
      successCondition: result < 2
      provider:
        prometheus:
          address: http://kube-prometheus-stack-1602-prometheus.prometheus.svc.cluster.local:
9090 # Nome DNS para o meu Prometheus, encontre o seu com kubectl describe svc ...
          query: |
            scalar(
              sum(kube_pod_container_status_restarts_total{namespace="default",
container="flaky-update"}) -
              sum(kube_pod_container_status_restarts_total{namespace="default",
container="flaky-update"} offset 2m)
            )
```

Crie um *AnalysisTemplate* para a aplicação Ping-pong que acompanhará o uso de CPU de todos os contêineres no *namespace*.

Outras estratégias de implantação

O Kubernetes suporta a estratégia de Recriação (*Recreate*), que derruba os *Pods* anteriores e substitui tudo pelo atualizado. O Argo Rollouts suporta a estratégia *BlueGreen*, na qual uma nova versão é executada lado a lado com a nova, mas o tráfego é alternado entre as duas em um determinado ponto.

Nossa aplicação de tarefas (*todo*) poderia ter um campo "Concluído" (*Done*) para tarefas que já foram feitas. Deve ser uma requisição PUT para */todos/*.